

Table des matières

Corrigé 07 — Sous-requêtes, corrélation et ensembles	1
Questions courtes	1
Questions pratiques	3
Diagnostics	7
Questions à développer	9
Mises en situation	10

Corrigé 07 — Sous-requêtes, corrélation et ensembles

Corrigé des exercices de Exercices_07.md. La numérotation est la même.

Questions courtes

1. Par exemple « quels produits n'ont jamais été vendus ? » et « quels clients ont acheté A mais pas B ? ». Ce qui leur manque : elles portent sur une absence ou sur une comparaison entre deux ensembles, alors qu'une agrégation ne sait décrire que ce qui est présent dans la table qu'elle parcourt.

Théorie : Theorie_07.md, section 1.

2. La sous-requête scalaire renvoie une seule valeur et vit partout où une valeur est attendue (SELECT, WHERE, HAVING). La sous-requête de table renvoie des lignes et des colonnes et vit dans le FROM, avec un alias obligatoire. La sous-requête de liste renvoie une colonne et plusieurs lignes, et s'emploie avec IN, ANY, ALL ou EXISTS.

Théorie : Theorie_07.md, section 2.

3. Elle vaut NULL. La comparaison avec NULL ne vaut ni vrai ni faux mais inconnu, donc la ligne ne passe pas le filtre et disparaît du résultat, sans erreur ni avertissement.

Théorie : Theorie_07.md, section 2.

4. Le moteur lève une erreur à l'exécution. C'est moins dangereux parce qu'une erreur se voit : elle arrête le traitement et vous force à corriger. Le cas de la question 3 produit un résultat plausible mais faux, que personne ne remarque.

Théorie : Theorie_07.md, section 2.

5. Elle référence une colonne d'une table déclarée dans la requête englobante, pas dans la sous-requête elle-même. Concrètement, on voit un alias qui n'est défini nulle part à l'intérieur des parenthèses.

Théorie : Theorie_07.md, section 3.

6. Parce que les moteurs modernes décorrèlent une partie de ces requêtes : un EXISTS ou un IN corrélié placé dans le WHERE devient une semi-jointure avant l'exécution, sans boucle. La règle n'est pas générale pour autant. Une sous-requête scalaire corréliée placée dans le SELECT reste un SubPlan, relancé pour chaque ligne, et c'est au rédacteur de la réécrire en jointure agrégée s'il veut autre chose. D'où la vraie réponse : ni croire le mythe, ni croire son contraire, mais lire le plan d'exécution et mesurer. La lenteur se constate, elle ne se devine pas.

Théorie : Theorie_07.md, section 3.

7. Non, le résultat serait identique : EXISTS ne regarde jamais les colonnes renvoyées, seulement s'il existe au moins une ligne. La convention SELECT 1 sert à dire au lecteur qu'on ne s'intéresse pas au contenu de la sous-requête.

Théorie : Theorie_07.md, section 4.

8. La semi-jointure filtre la table du FROM en vérifiant qu'au moins une correspondance existe dans une autre table, sans rien ramener de celle-ci : c'est EXISTS. L'anti-jointure garde au contraire les lignes de la table du FROM qui n'ont aucune correspondance : c'est NOT EXISTS.

Théorie : Theorie_07.md, section 4.

9. Parce que EXISTS s'arrête dès la première ligne trouvée, alors que COUNT(*) doit parcourir toutes les lignes correspondantes avant de pouvoir comparer le total à zéro.

Théorie : Theorie_07.md, section 4.

10. `x NOT IN (a, b, NULL)` se développe en `x <> a AND x <> b AND x <> NULL`. Le dernier terme vaut inconnu, jamais vrai. Or un AND dont un terme est inconnu ne peut plus valoir vrai : au mieux il vaut inconnu. Aucune ligne ne satisfait donc le filtre.

Théorie : Theorie_07.md, section 5.

11. Parce qu'une erreur de syntaxe arrête la requête et se corrige. Ici, la requête s'exécute et renvoie zéro ligne, ce qui est une réponse crédible à « qu'est-ce qui manque ». Le résultat faux part en production sans que rien ne le signale.

Théorie : Theorie_07.md, section 5.

12. NOT EXISTS, NOT IN, et le LEFT JOIN suivi d'un test IS NULL sur la clé de la table de droite. Par défaut on écrit NOT EXISTS : c'est la seule des trois qui soit à la fois correcte quelles que soient les valeurs et lisible dans son intention.

Théorie : Theorie_07.md, sections 4 et 5.

13. `x IN (...)` équivaut à `x = ANY (...)`. `x NOT IN (...)` équivaut à `x <> ALL (...)`.

Théorie : Theorie_07.md, section 5.

14. $x > ALL$ (...) sur un ensemble vide vaut vrai, parce qu'aucun élément ne vient contredire l'affirmation. $x > ANY$ (...) vaut faux, parce qu'aucun élément ne vient la confirmer. La conséquence pratique : une requête bâtie sur ALL laisse passer toutes les lignes le jour où sa sous-requête ne renvoie plus rien.

Théorie : Theorie_07.md, section 5.

15. UNION, UNION ALL, INTERSECT, EXCEPT. Seul UNION ALL ne dédoublonne pas.

Théorie : Theorie_07.md, section 6.

16. Même nombre de colonnes, types compatibles colonne par colonne, et l'ordre des colonnes fait foi. Ce sont les noms du premier SELECT qui nomment le résultat ; les noms des suivants sont ignorés.

Théorie : Theorie_07.md, section 6.

17. Parce que le dédoublonnage oblige le moteur à trier ou à hacher tout le résultat, ce qui est souvent l'opération la plus chère de la requête, et parce qu'on n'a le plus souvent aucun doublon à supprimer. On passe à UNION uniquement quand on sait qu'il y a des doublons et qu'on veut vraiment s'en débarrasser.

Théorie : Theorie_07.md, section 6.

18. Il autorise la sous-requête à référencer les tables qui la précèdent dans la clause FROM. Sans lui, une sous-requête du FROM est évaluée dans le vide, avant les autres tables, et ne peut donc rien en lire.

Théorie : Theorie_07.md, section 7.

19. Le ON true est une formalité de syntaxe : la condition de rapprochement vit déjà dans le WHERE de la sous-requête, il ne reste rien à mettre dans le ON, mais la grammaire d'un JOIN en exige un. Le LEFT conserve les lignes de la table de gauche pour lesquelles la sous-requête ne renvoie rien ; sans lui, elles disparaissent du résultat.

Théorie : Theorie_07.md, section 7.

20. La fonction de fenêtrage avec ROW_NUMBER, puis filtrage du rang : le meilleur choix quand il s'agit vraiment d'un classement, parce qu'elle ne fait qu'un passage. QUALIFY sur l'entrepôt : la même chose en plus court, quand on est sur cette plateforme. LATERAL : le meilleur choix quand la sous-requête fait autre chose qu'un classement, par exemple aller chercher la dernière valeur connue avant une date.

Théorie : Theorie_07.md, section 7.

Questions pratiques

P1. [OLTP]

```
SELECT p.name, p.listprice
FROM   production.product AS p
WHERE  NOT EXISTS (SELECT 1
                   FROM   sales.salesorderdetail AS d
                   WHERE  d.productid = p.productid)
ORDER BY p.name;
```

Environ deux cent trente-huit lignes. NOT EXISTS plutôt que NOT IN : même si salesorderdetail.productid n'est pas nullable aujourd'hui, la règle ne dépend pas d'une vérification qu'il faudrait refaire à chaque évolution du schéma.

Théorie : Theorie_07.md, section 4.

P2. [OLAP] Le piège de l'énoncé est qu'un client sans aucune commande satisfait aussi la condition. Si l'on veut les clients qui ont commandé mais plus depuis 2013, il faut l'exiger explicitement.

```
SELECT c.CustomerKey
FROM   DimCustomer AS c
WHERE  EXISTS (SELECT 1 FROM FactInternetSales AS f
              WHERE f.CustomerKey = c.CustomerKey)
AND    NOT EXISTS (SELECT 1 FROM FactInternetSales AS f
                  WHERE f.CustomerKey = c.CustomerKey AND f.OrderDateKey > 20130101);
```

Sans le premier EXISTS, la liste mélange les clients inactifs et les clients qui n'ont jamais rien acheté, deux populations qui n'appellent pas la même action commerciale.

Théorie : Theorie_07.md, sections 4 et 5.

P3. [OLTP]

```
-- Version corrélée.
SELECT p.name, p.listprice,
       (SELECT AVG(q.listprice) FROM production.product AS q
        WHERE q.productssubcategoryid = p.productssubcategoryid) AS moyenne_sc,
       p.listprice - (SELECT AVG(q.listprice) FROM production.product AS q
                     WHERE q.productssubcategoryid = p.productssubcategoryid) AS ecart
FROM   production.product AS p
WHERE  p.productssubcategoryid IS NOT NULL;

-- Version fenêtre.
SELECT p.name, p.listprice,
       AVG(p.listprice) OVER (PARTITION BY p.productssubcategoryid) AS moyenne_sc,
       p.listprice - AVG(p.listprice) OVER (PARTITION BY p.productssubcategoryid) AS ecart
```

```
FROM production.product AS p
WHERE p.productsubcategoryid IS NOT NULL;
```

On garde la fenêtre. Elle exprime la même chose en un seul passage, elle ne répète pas le calcul deux fois, et elle reste juste si l'on ajoute une colonne. La version corrélée a un mérite pédagogique et rien d'autre ici.

Théorie : Theorie_07.md, section 3.

P4. [OLAP]

```
WITH achats AS (
  SELECT f.CustomerKey, c.EnglishProductCategoryName AS categorie
  FROM FactInternetSales AS f
  JOIN DimProduct AS p ON p.ProductKey = f.ProductKey
  JOIN DimProductSubcategory AS sc ON sc.ProductSubcategoryKey = p.ProductSubcategoryKey
  JOIN DimProductCategory AS c ON c.ProductCategoryKey = sc.ProductCategoryKey
)
SELECT CustomerKey FROM achats WHERE categorie = 'Bikes'
INTERSECT
SELECT CustomerKey FROM achats WHERE categorie = 'Clothing';
```

Sans opérateur d'ensemble, avec deux EXISTS sur la même table de faits. La CTE ne vit que le temps de la requête qui la déclare : la seconde écriture doit donc la déclarer à nouveau, sans quoi DuckDB répond Table with name achats does not exist, mesuré.

```
WITH achats AS (
  SELECT f.CustomerKey, c.EnglishProductCategoryName AS categorie
  FROM FactInternetSales AS f
  JOIN DimProduct AS p ON p.ProductKey = f.ProductKey
  JOIN DimProductSubcategory AS sc ON sc.ProductSubcategoryKey = p.ProductSubcategoryKey
  JOIN DimProductCategory AS c ON c.ProductCategoryKey = sc.ProductCategoryKey
)
SELECT c.CustomerKey
FROM DimCustomer AS c
WHERE EXISTS (SELECT 1 FROM achats AS a
              WHERE a.CustomerKey = c.CustomerKey AND a.categorie = 'Bikes')
AND EXISTS (SELECT 1 FROM achats AS a
            WHERE a.CustomerKey = c.CustomerKey AND a.categorie = 'Clothing');
```

Les deux rendent les mêmes 3 172 clients. La seconde se lit mieux dès qu'on veut ajouter une troisième condition, et elle permet de garder d'autres colonnes de DimCustomer. L'INTERSECT est plus concis mais ne renvoie que la clé.

Théorie : Theorie_07.md, section 6.

P5. [OLTP]

```

SELECT sc.name AS sous_categorie, top.name, top.listprice
FROM   production.productssubcategory AS sc
LEFT   JOIN LATERAL (SELECT p.name, p.listprice
                      FROM   production.product AS p
                      WHERE  p.productssubcategoryid = sc.productssubcategoryid
                      ORDER  BY p.listprice DESC
                      LIMIT  2) AS top ON true
ORDER  BY sous_categorie, top.listprice DESC;

```

Le LEFT est ce qui répond à « en conservant les sous-catégories qui n'ont aucun produit ». Les lignes correspondantes porteraient alors des NULL dans les colonnes de top.

Au conditionnel, parce que sur cette base il n'y en a aucune : les 37 sous-catégories ont toutes au moins un produit, et la requête rend 37 groupes avec ou sans le LEFT. C'est exactement la situation qui fait tomber les gens, et elle mérite d'être vue ici plutôt que dans six mois. Une requête dont le LEFT ne change rien aujourd'hui n'est pas une requête où le LEFT est inutile : c'est une requête où la donnée ne l'a pas encore exercé. Le jour où une sous-catégorie naît sans produit, la version sans LEFT la perd en silence. Écrivez donc le LEFT d'après la question posée, jamais d'après ce que les données contiennent au moment où vous écrivez. Le diagnostic E5 montre le même mécanisme sur un terrain où la perte est immédiate, 17 groupes au lieu de 290.

Théorie : Theorie_07.md, section 7.

P6. [OLAP]

```

SELECT sc.EnglishProductSubcategoryName AS sous_categorie,
       p.EnglishProductName, p.ListPrice
FROM   DimProduct AS p
JOIN   DimProductSubcategory AS sc ON sc.ProductSubcategoryKey = p.ProductSubcategoryKey
QUALIFY ROW_NUMBER() OVER (PARTITION BY sc.ProductSubcategoryKey
                           ORDER BY p.ListPrice DESC) <= 2;

```

Ce que la version LATERAL fait de plus : elle part des sous-catégories, donc elle peut conserver celles qui n'ont aucun produit. Celle-ci part des produits et ne peut pas les inventer.

Théorie : Theorie_07.md, sections 7 et 8.

P7. Sur l'entrepôt, le compte puis le détail :

```

-- Le compte.
SELECT COUNT(*) AS lignes_orphelines
FROM   FactInternetSales AS f
WHERE  NOT EXISTS (SELECT 1 FROM DimProduct AS p WHERE p.ProductKey = f.ProductKey);

-- Les dix clés fautives les plus fréquentes.
SELECT f.ProductKey, COUNT(*) AS nb_lignes

```

```

FROM FactInternetSales AS f
WHERE NOT EXISTS (SELECT 1 FROM DimProduct AS p WHERE p.ProductKey = f.ProductKey)
GROUP BY f.ProductKey
ORDER BY nb_lignes DESC
LIMIT 10;

```

Le passage du compte au détail est ce qui rend le contrôle exploitable : un nombre dit qu'il y a un problème, la liste des clés dit par où commencer.

Théorie : Theorie_07.md, section 4.

P8. [OLTP]

```

-- Version fautive : renvoie 0.
SELECT COUNT(*)
FROM production.productssubcategory AS sc
WHERE sc.productscategoryid NOT IN (SELECT p.productscategoryid
                                   FROM production.product AS p);

-- Correction 1 : écarter les NULL de la sous-requête.
SELECT COUNT(*)
FROM production.productssubcategory AS sc
WHERE sc.productscategoryid NOT IN (SELECT p.productscategoryid
                                   FROM production.product AS p
                                   WHERE p.productscategoryid IS NOT NULL);

-- Correction 2 : La bonne, celle qui ne demande rien à savoir sur les données.
SELECT COUNT(*)
FROM production.productssubcategory AS sc
WHERE NOT EXISTS (SELECT 1 FROM production.product AS p
                  WHERE p.productscategoryid = sc.productscategoryid);

```

Les deux corrections donnent le même compte. La première exige de savoir que la colonne est nullable, ce qu'il faudrait vérifier à chaque évolution du schéma. La seconde est juste par construction. C'est pour cela que la règle est NOT EXISTS, et non « NOT IN avec précaution ».

Théorie : Theorie_07.md, section 5.

Diagnostics

E1. Deux défauts se cumulent.

Le premier est le NOT IN sur une sous-requête, avec le risque NULL habituel. Le second est plus subtil et c'est lui qui rend le résultat faux même sans aucun NULL : le filtre orderqty > 5 restreint la liste des produits vendus aux seules lignes de plus de cinq unités. Un produit vendu une seule fois, en deux exemplaires, sera donc déclaré « jamais vendu ». La condition n'a rien à faire dans une anti-jointure qui teste l'existence.

```
SELECT p.name
FROM   production.product AS p
WHERE  NOT EXISTS (SELECT 1 FROM sales.salesorderdetail AS d
                   WHERE d.productid = p.productid);
```

La leçon générale : dans une anti-jointure, toute condition ajoutée à la sous-requête rétrécit l'ensemble de référence, donc élargit le résultat. Il faut se demander à chaque fois si c'est voulu.

Théorie : Theorie_07.md, sections 4 et 5.

E2. La sous-requête n'est pas corrélée : rien à l'intérieur ne fait référence à c. Elle est donc calculée une fois et la même valeur, le total général des commandes, s'affiche sur chaque ligne.

```
SELECT c.customerid,
       (SELECT COUNT(*) FROM sales.salesorderheader AS h
        WHERE h.customerid = c.customerid) AS nb_commandes
FROM   sales.customer AS c;
```

Théorie : Theorie_07.md, section 3.

E3. Les types ne correspondent pas colonne par colonne : le premier SELECT empile un entier puis un texte, le second un texte puis un entier. PostgreSQL refuse.

Le même genre d'erreur passerait inaperçu si les deux colonnes inversées avaient des types compatibles, par exemple deux textes ou deux entiers. La requête s'exécuterait alors sans un mot, en empilant des valeurs qui n'ont rien à voir sous le nom de colonne du premier SELECT. Mesuré sur l'entrepôt avec deux entiers, la clé d'un produit et son stock de sécurité inversés : 210 et 500 d'un côté, 500 et 210 de l'autre, sans erreur. C'est pourquoi il faut lire l'ordre des colonnes plutôt que se fier à leurs noms.

Une bonne réponse nomme la parade qui existe sur l'entrepôt : UNION BY NAME, qui aligne les colonnes par leur nom. Écrivez ainsi, la requête du collègue passerait sur DuckDB et rangerait chaque valeur à sa place, mesuré ; PostgreSQL ne connaît pas cette forme et la refuse par une erreur de syntaxe. Snowflake l'a aussi, d'après sa documentation.

Théorie : Theorie_07.md, section 6.

E4. Deux raisons distinctes.

La première tient au NOT IN : si DimProduct.ProductKey contenait un NULL, le résultat serait zéro quelles que soient les données, donc le zéro ne prouve rien.

La seconde tient au filtre Status = 'Current'. La sous-requête ne liste que les versions courantes de la dimension. Une ligne de fait pointant une version historique parfaitement valide serait comptée comme orpheline, ou l'inverse selon l'écriture. Le contrôle ne mesure donc pas ce que son auteur croit mesurer.

Un contrôle correct porte sur la dimension entière et s'écrit avec NOT EXISTS.

Théorie : Theorie_07.md, section 4.

E5. La virgule et LATERAL produisent une jointure interne : un employé dont la sous-requête ne renvoie aucune ligne disparaît du résultat. Les 273 employés absents n'ont jamais pris de commande, ce qui est la situation de la grande majorité d'entre eux : sur 290 employés, 17 seulement sont vendeurs.

```
SELECT e.businessentityid, top.salesorderid, top.totaldue
FROM   humanresources.employee AS e
LEFT   JOIN LATERAL (SELECT h.salesorderid, h.totaldue FROM sales.salesorderheader AS h
                     WHERE h.salespersonid = e.businessentityid
                     ORDER BY h.totaldue DESC LIMIT 3) AS top ON true;
```

Attendu : 290 employés couverts au lieu de 17, les 273 autres apparaissant avec des NULL dans les colonnes de commande.

Reste la question qui compte, celle de la disparition inaperçue. Un rapport de top N ne montre que des lignes pleines : il n'y a pas de trou visible, pas de cellule vide, rien qui signale l'absence. Le lecteur voit dix-sept employés avec leurs meilleures commandes, et rien ne lui dit qu'il en manque 273. Le seul contrôle qui l'aurait attrapé est celui qu'on oublie toujours : compter les groupes en entrée et les groupes en sortie, et vérifier qu'ils coïncident. C'est le même geste que le contrôle d'exactitude du Cours 12.

C'est un défaut typique d'un rapport de pilotage : les groupes vides sont précisément ceux dont la direction voudrait entendre parler, et ce sont eux qui s'effacent.

Théorie : Theorie_07.md, section 7.

Questions à développer

A1. Une bonne réponse défend la règle par le coût de lecture. Une jointure déclare une relation entre deux tables, elle se lit de gauche à droite et le moteur la traite d'un bloc. Une sous-requête introduit un niveau d'imbrication qui se lit de l'intérieur vers l'extérieur, à contresens du sens de lecture naturel. À résultat égal, la jointure demande donc moins d'effort au lecteur suivant, et c'est ce lecteur qui paiera vos choix.

Le contre-exemple attendu : la semi-jointure. EXISTS exprime « au moins une correspondance » sans risquer de multiplier les lignes de la table de gauche, là où une jointure sur une relation un-vers-plusieurs duplique les lignes et oblige à un DISTINCT correctif. Le DISTINCT masque alors l'intention et coûte un dédoublonnage. On accepte aussi comme contre-exemple la sous-requête scalaire de contexte quand la valeur est vraiment globale.

Est également acceptable une réponse qui conteste la règle en soulignant qu'elle privilégie la lisibilité sur la performance, à condition de dire pourquoi ce choix est le bon par défaut.

Théorie : Theorie_07.md, section 1.

A2. Le point à faire passer : le fait qu’aucune requête ne soit fausse aujourd’hui est une propriété des **données**, pas du code. Elle tient tant qu’aucune colonne concernée ne reçoit de NULL. Or c’est exactement le genre de propriété qui change sans prévenir, quand une colonne devient optionnelle, quand une source ajoute des lignes incomplètes, ou quand un `LEFT JOIN` en amont introduit des NULL là où il n’y en avait pas.

Une bonne réponse ajoute que le mode de défaillance est silencieux : la requête ne casse pas, elle renvoie zéro ligne. Le jour où le problème arrive, il ne se manifeste pas par une alerte mais par un rapport qui semble dire que tout va bien. Deux jours de migration achètent la disparition d’une classe entière de pannes invisibles, ce qui est un excellent rapport.

Théorie : Theorie_07.md, section 5.

A3. `NOT EXISTS` compare selon une **condition** que vous écrivez, généralement une égalité de clé, et il filtre les lignes de la table du `FROM` en conservant toutes leurs colonnes. `EXCEPT` compare des **lignes entières**, colonne par colonne et dans l’ordre, entre deux résultats de même forme, et il dédouble le résultat.

Le cas où le remplacement change le résultat : dès que les deux côtés n’ont pas la même liste de colonnes. Si l’on ajoute une colonne au premier `SELECT` d’un `EXCEPT`, deux lignes qui différaient seulement par cette colonne cessent d’être considérées comme identiques, et la différence d’ensembles change. Un `NOT EXISTS` sur la clé n’aurait pas bougé. Le dédoublement d’`EXCEPT` est un second point de divergence : si la table de départ (celle du `FROM` pour `NOT EXISTS`, celle du premier `SELECT` pour `EXCEPT`) contient des doublons légitimes, `EXCEPT` les réduit à un, `NOT EXISTS` les conserve tous.

Théorie : Theorie_07.md, sections 4 et 6.

A4. L’objection est partiellement fondée et il faut le dire d’abord. `LATERAL` exprime bien une exécution pour chaque ligne de la table de gauche, et si cette table compte des millions de lignes pendant que la sous-requête parcourt une grosse table, le coût est réel et une fonction de fenêtrage fera mieux.

Mais l’objection généralise à tort, pour trois raisons. D’abord, `LATERAL` sert typiquement à partir d’une petite table de gauche, un référentiel de catégories ou d’agences, ce qui rend le nombre d’itérations dérisoire. Ensuite, le moteur optimise parfois ce motif ; le plan d’exécution le dit, et il faut le lire plutôt que le supposer, puisque la décorrélation ne joue pas partout, comme le labo l’a montré sur la scalaire du `SELECT`. Enfin, `LATERAL` exprime des choses qu’aucune fenêtrage n’exprime : dès que la sous-requête fait autre chose qu’un classement, par exemple chercher la dernière valeur antérieure à une date, la fenêtrage n’est plus une alternative.

Une bonne réponse conclut sur la méthode plutôt que sur le verdict : on écrit ce qui exprime la question, on lit le plan, et on ne remplace que si la mesure le justifie.

Théorie : Theorie_07.md, section 7.

Mises en situation

M1. Réponse défendable.

La première liste est une anti-jointure avec condition de date : NOT EXISTS sur livraison, avec le filtre de date **dans** la sous-requête. Le point qui distingue une bonne réponse d'une réponse approximative est celui du diagnostic E1 : mettre le filtre de date dans le WHERE extérieur ne donnerait pas du tout la même chose. Il faut aussi décider si les abonnés n'ayant jamais reçu de livraison entrent dans la liste, et le dire.

La deuxième liste est une différence d'ensembles : les abonnés d'une période moins ceux d'une autre, avec EXCEPT ou avec un couple EXISTS et NOT EXISTS. Le piège NULL guette précisément si l'on écrit cette différence avec NOT IN sur une liste d'identifiants issue d'une jointure, où un LEFT JOIN en amont peut avoir introduit des NULL qui n'existaient dans aucune table de départ. C'est la réponse attendue à la question posée.

La troisième liste est un top N par groupe. Sur PostgreSQL, LATERAL ou fenêtre plus CTE ; sur l'entrepôt, QUALIFY. Une bonne réponse signale qu'il faut conserver les régions sans abonné actif, donc LEFT JOIN LATERAL, faute de quoi une région en difficulté disparaît du rapport, ce qui est l'inverse du besoin.

Théorie : Theorie_07.md, sections 4, 6 et 7.

M2. Une bonne batterie contient au moins trois contrôles.

Les entités perdues : NOT EXISTS de l'ancien vers le nouveau, sur la clé métier, pour chacune des trois tables. Un résultat non vide nomme ce qui a disparu. Les entités apparues : la même chose dans l'autre sens, qui révèle les doublons créés par la migration ou les lignes fabriquées. Les orphelins : NOT EXISTS de consultation vers animal et de animal vers propriétaire, dans le nouveau système, pour vérifier que les liens ont suivi.

La partie qui distingue une bonne réponse est la seconde. Un résultat vide ne prouve pas que rien n'a été perdu : il prouve seulement que chaque clé présente d'un côté l'est de l'autre. Il ne dit rien du **contenu** des lignes. Une migration peut conserver toutes les clés en tronquant les noms, en décalant les dates d'un fuseau, ou en arrondissant les montants. Le contrôle manquant est une comparaison colonne à colonne des lignes appariées, qu'EXCEPT fait très bien puisqu'il compare des lignes entières. C'est l'usage où EXCEPT bat NOT EXISTS, et le lien avec la question A3.

Théorie : Theorie_07.md, section 6.

M3. Réponse défendable.

Les machines jamais louées du mois sont une anti-jointure avec filtre de date dans la sous-requête, et il faut conserver les agences sans aucune location, donc partir de agence et non de location.

Pour les trois machines les plus rentables par agence, l'arbitrage attendu est le suivant. Sur l'entrepôt, QUALIFY sur une fenêtre : c'est un classement pur, la fenêtre est faite pour ça, et la table des locations est grosse. Sur PostgreSQL, la fenêtre plus CTE pour la même raison ; LATERAL ne se justifierait que si la table de gauche, les agences, est petite et si l'on veut conserver celles qui n'ont rien loué. Une bonne réponse mentionne ce dernier point, parce qu'une agence sans location est précisément une agence dont la direction veut entendre parler.

Les catégories présentes ici et absentes là sont un EXCEPT entre deux agences, ou une généralisation par anti-jointure si la comparaison doit se faire entre toutes les paires d'agences. Signaler que la version « toutes les paires » ne s'écrit pas avec EXCEPT est un bon point.

Ce qu'il faut refuser de livrer : la notion de machine « la plus rentable » n'est pas définie. Le chiffre d'affaires de location, la marge après entretien et le taux d'utilisation donnent trois classements différents, et une machine louée tous les jours à bas prix peut être plus rentable qu'une machine louée trois fois cher. La

question à poser à la direction porte donc sur la définition de la rentabilité, et sur ce qu'elle compte faire du classement. Livrer un chiffre d'affaires en l'appelant rentabilité serait une faute que personne ne verrait avant qu'une décision soit prise dessus.

Théorie : Theorie_07.md, sections 4, 6 et 7.